

Kapitel 1: Software, Tools und Programmiersprache

Einige dieser Aspekte haben wir bereits gemeinsam eingerichtet.

Lest euch die folgenden Inhalte dennoch noch einmal in Ruhe durch.

Vor allem für den Fall, dass ihr zu Hause etwas für den Kurs erledigen oder programmieren möchtet, zum Beispiel in Vorbereitung auf die Leistungskontrollen, sind diese Hinweise hilfreich, damit ihr euren PC daheim entsprechend einrichten könnt. In Absprache mit einem Informatiklehrer ist es natürlich auch immer möglich, das Computerkabinett der Schule zu benutzen.

1.1. Programmierumgebung

Als **Programmierumgebung** (auch: IDE, aus dem Englischen: integrated development environment) nutzen wir **Visual Studio Code**. Dieser Editor steht für die meisten Betriebssysteme zur Verfügung. Heruntergeladen werden kann dieser Editor von folgender Website:

<https://code.visualstudio.com/>

Visual Studio Code kann - mit ein paar Einschränkungen im Funktionsumfang - auch im **Browser** genutzt werden, beispielsweise für den Fall, dass ihr zu Hause nur mit einem Tablet arbeiten wollt oder keine Programme installieren könnt.

<https://vscode.dev/>

Die Software ist **kostenfrei**. Visual Studio Code basiert auf dem quelloffenen Projekt **Code - OSS**; die von Microsoft bereitgestellte Version von Visual Studio Code wird jedoch unter einer eigenen Microsoft-Produktlizenz veröffentlicht.

1.2. Python zu Hause installieren

In der Schule ist Python auf den Rechnern bereits installiert. Dort müsst ihr Python also nicht selbst einrichten.

Wenn ihr zu Hause an euren eigenen Geräten arbeiten möchtet, müsst ihr Python gegebenenfalls selbst installieren. Achtet dabei darauf, Python nur aus vertrauenswürdigen Quellen zu installieren.

Windows

Unter Windows könnt ihr Python über den **Microsoft Store** installieren. Sucht dort nach **Python** und achtet darauf, dass das Paket von der **Python Software Foundation** stammt. Python ist kostenlos. Wenn ihr zur Zahlung aufgefordert werdet, habt ihr nicht das richtige Paket ausgewählt.

Alternativ könnt ihr den Python Install Manager auch von der offiziellen Python-Website herunterladen:

<https://www.python.org/downloads/>

Nach der Installation könnt ihr prüfen, ob Python richtig installiert wurde:

1. Öffnet das **Startmenü**.
2. Tippt **Terminal** oder **PowerShell** ein.
3. Öffnet die angezeigte App.
4. Gebt den folgenden Befehl ein und drückt **[ENTER]** :

```
python --version
```

Wenn eine Versionsnummer angezeigt wird, zum Beispiel **Python 3.13.2** , wurde Python erkannt.

macOS

Unter macOS installiert ihr Python über die offizielle Python-Website. Öffnet dazu die folgende Seite und ladet den aktuellen **macOS installer** herunter:

<https://www.python.org/downloads/macos/>

Nach der Installation könnt ihr prüfen, ob Python richtig installiert wurde:

1. Öffnet die **Spotlight-Suche** mit **[CMD] + [LEERTASTE]** .
2. Tippt **Terminal** ein.
3. Öffnet die App **Terminal**.
4. Gebt den folgenden Befehl ein und drückt **[ENTER]** :

```
python3 --version
```

Wenn eine Versionsnummer angezeigt wird, zum Beispiel **Python 3.13.2** , wurde Python erkannt.

Startet Visual Studio Code nach der Installation von Python neu, damit die neue Python-Installation erkannt werden kann. Falls Python im Terminal funktioniert, aber in Visual Studio Code noch nicht erkannt wird, schließt Visual Studio Code vollständig und öffnet es danach erneut.

1.3. Erweiterungen in Visual Studio Code

Visual Studio Code ist in der Basis-Variante ein sehr leistungsfähiger **Texteditor**.

Mit Erweiterungen, die das Programm um zusätzliche Funktionen ergänzen, können wir den Texteditor noch mächtiger machen.

Für die Bearbeitung des Kurses benötigen wir unbedingt die nachfolgenden **Erweiterungen**.

- Python
- Jupyter
- vscode-pdf

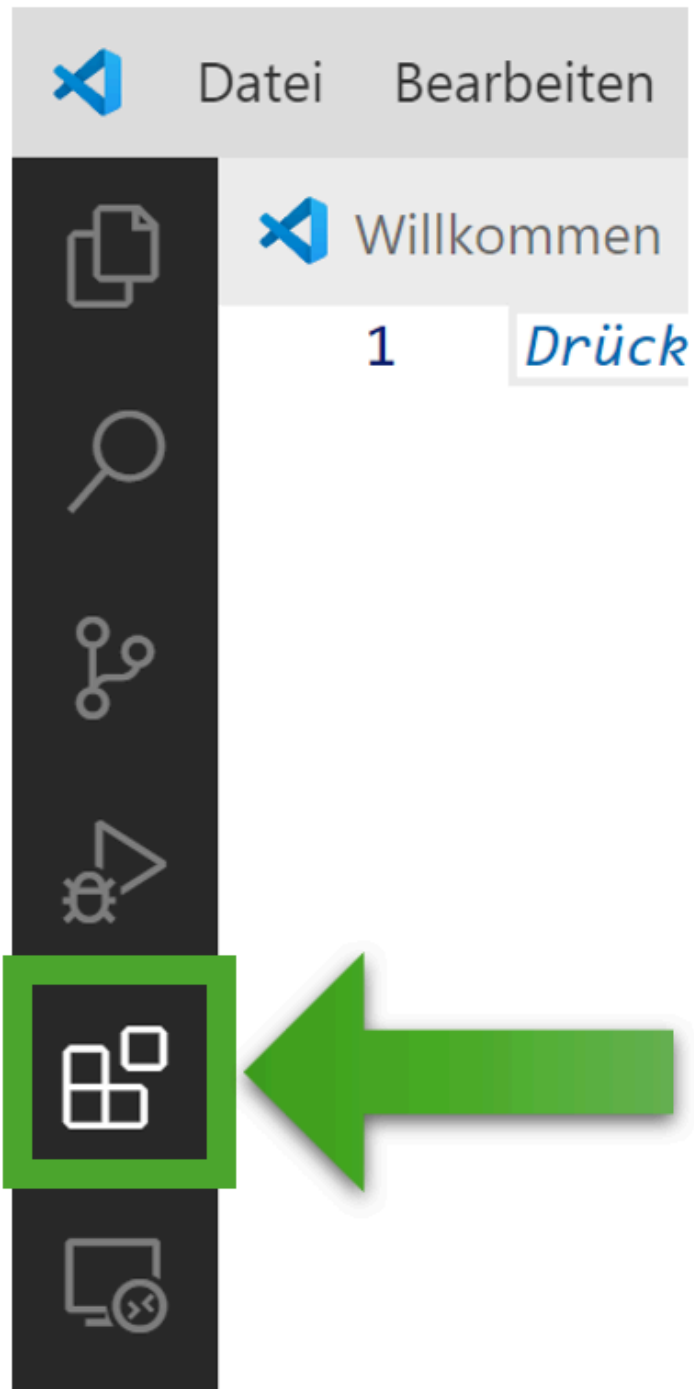
Optional empfohlen wird noch das deutsche Sprachpaket.

- German Language Pack for Visual Studio Code

Installation von Erweiterungen

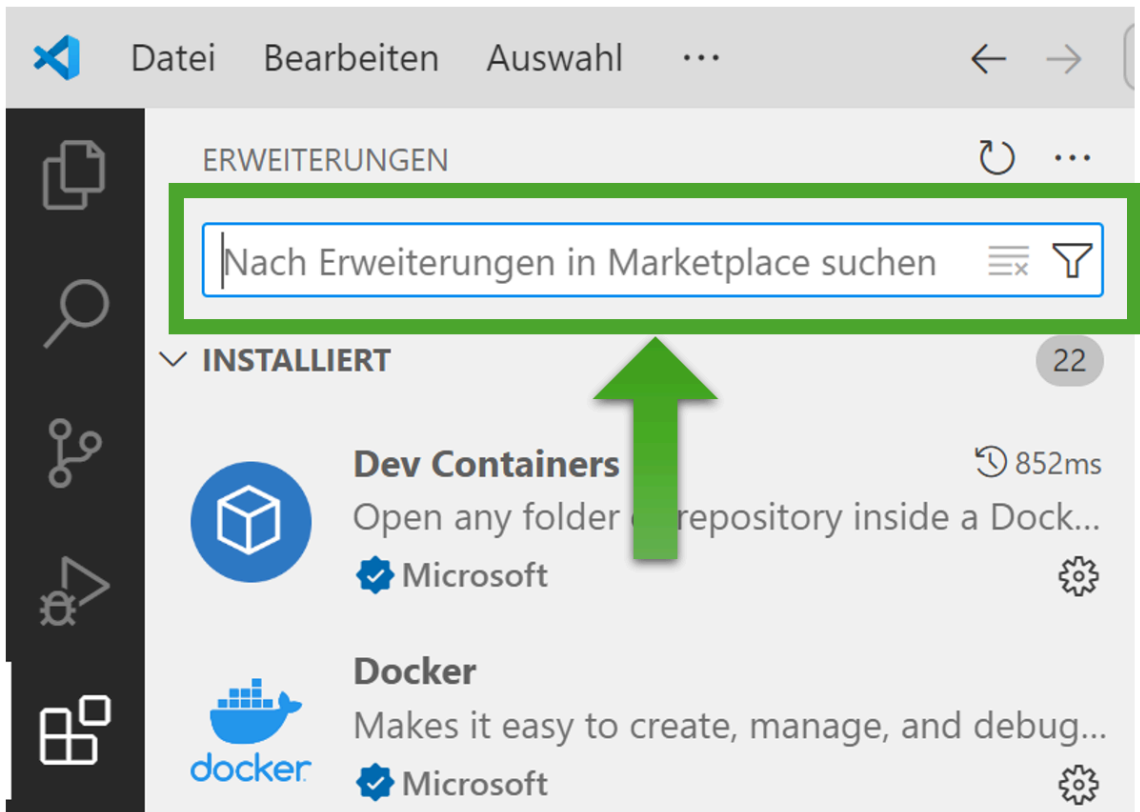
Erweiterungen können ganz einfach installiert werden.

Klickt dazu in der linken Menüleiste auf das Icon für die Erweiterungen. Das ist auf dem nachfolgenden Bild markiert.



In der Suchleiste könnt ihr die Erweiterung anhand des Namens suchen.

Hat man die richtige Erweiterung gefunden, klickt man nur noch auf **Installieren** und wartet den Installationsprozess **vollständig** ab.



Hilfsprogramm zur Installation

Damit die Installation schneller vonstattengeht und immer die richtigen Erweiterungen installiert werden, steht euch in der Schule ein Hilfsprogramm zur Verfügung.

Navigiert dazu im Schulnetzwerk zu folgender Datei:

```
K:\PYTHON_PACKAGES_BULK_INSTALL\PYTHON_PACKAGES_INSTALLER.bat
```

Klickt doppelt auf die Datei (die hat ein Zahnrad-Icon) und lasst die Installation im schwarzen Terminalfenster vollständig durchlaufen.

Damit sind alle benötigten (und ein paar weitere sinnvolle) Erweiterungen installiert.

Nach der Installation wird auffallen, dass Visual Studio Code **noch nicht auf Deutsch** umgestellt ist.

So stellt ihr Visual Studio Code auf Deutsch um: Nutzt in Visual Studio Code die Tastenkombination `[STRG] + [SHIFT] + [P]`. Im oberen Bereich öffnet sich ein Eingabefeld für die Befehlspalette. Gebt dort nun `Language` ein. Es erscheint die Option `Configure Display Language`. Klickt auf diese Option und wählt anschließend Deutsch als Anzeigesprache aus.

Alle Dateien im Ordner ...

`K:\PYTHON_PACKAGES_BULK_INSTALL`

... auf dem Schullaufwerk könnt ihr euch auch kopieren und auf LernSax hochladen. Oder optional steht das ganze im Respository als Download zur Verfügung im Ordner `00_BULK_INSTALL_VS_CODE`. Damit könnt ihr auch **zu Hause**, wenn ihr dort ein **Windows-System** nutzt, alle Erweiterungen automatisiert installieren lassen.

1.4. Kernel

Der **Kernel** ist das Programm, das den Code in einem Jupyter-Notebook-Dokument ausführt.

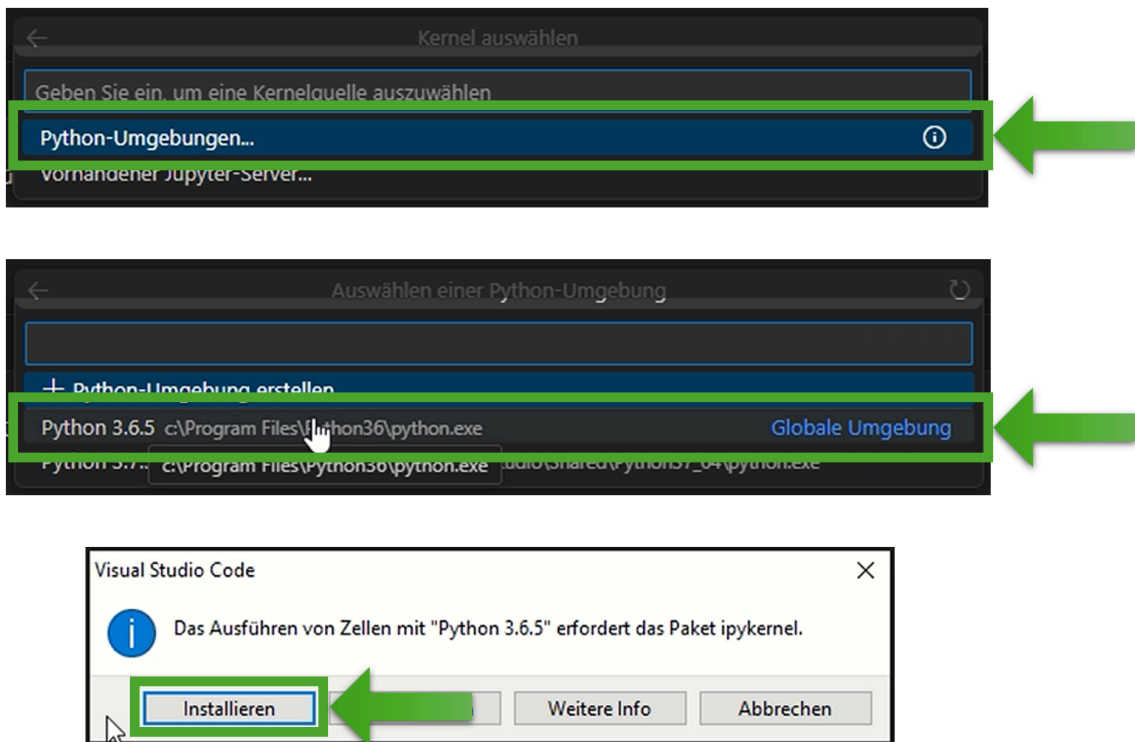
Ohne einen passenden Python-Kernel könnte Jupyter keinen Python-Code verarbeiten - das Notebook wäre dann nur ein Dokument mit Text und Code.

Bevor wir Code-Zeilen ausführen können, müssen wir daher eine Python-Umgebung als Kernel auswählen.

Falls dafür noch ein Zusatzpaket wie `ipykernel` fehlt, bietet Visual Studio Code dessen Installation automatisch an.

Bei der ersten Ausführung eines Code-Blocks erscheint im oberen Bereich ein entsprechender Dialog.

Der nachfolgende Screenshot zeigt euch, was ihr dabei jeweils anklicken müsst:



Die Installation dauert einen **kurzen Moment**. Wundert euch nicht darüber. Das hat in der Regel alles seine Richtigkeit.

1.5. Testen

Im Ordner zum Kurs findet ihr die Datei `00_einrichtung.py` - eine erste kleine Python-Datei.

Öffnet diese Datei!

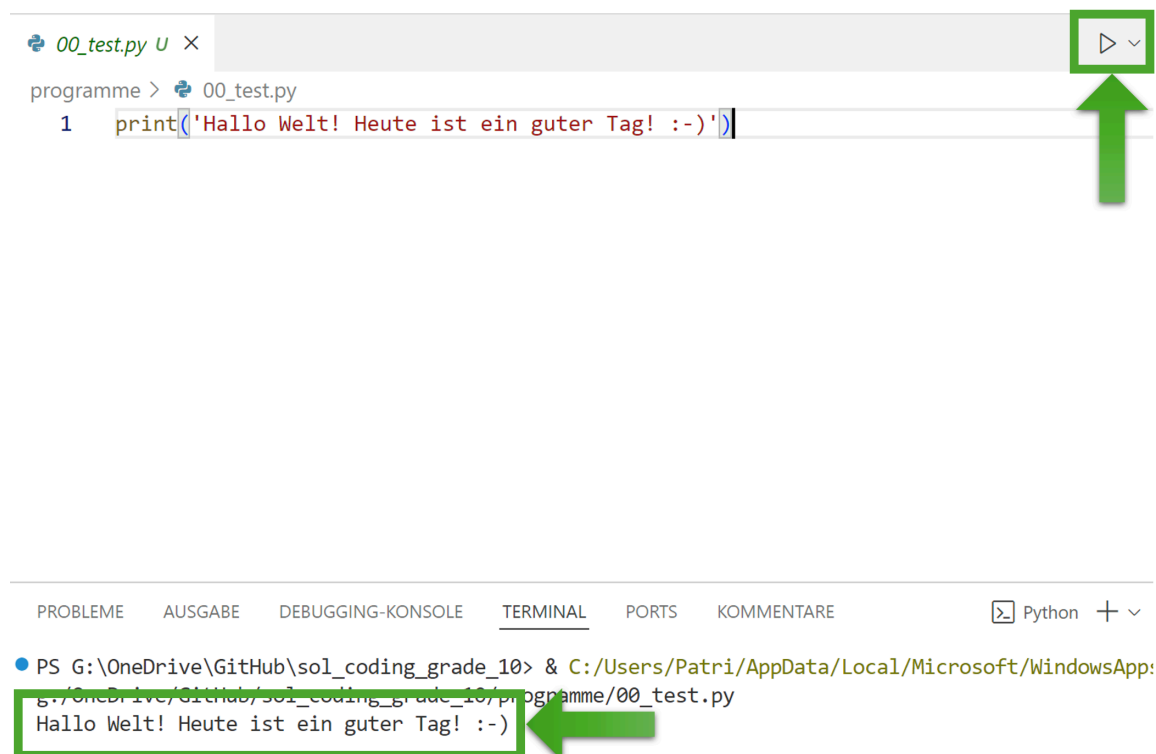
Im oberen rechten Bereich sollte ein Play-Button angezeigt werden. Mit diesem könnt ihr das Programm ausführen.

Im unteren Bereich öffnet sich dann das Terminal.

Dort wird eine Ausgabe erzeugt. Zu lesen sollte dort `Hallo Welt! Heute ist ein guter Tag! :-)` sein.

Probiert das aus!

Der nachfolgende Screenshot zeigt noch einmal, wo ihr klicken müsst:



Außerdem findet ihr eine Datei mit dem Namen `00_einrichtung.ipynb`.

Das ist ein **erstes Jupyter-Notebook-Dokument**.

Öffnet auch dieses und klickt neben dem Code-Block (auf der linken Seite) auf den Play-Button. **Direkt unter dem Block** sollte nun ebenfalls die Ausgabe erscheinen.

1.6. Warum Python?

Python ist eine **weit verbreitete, einfach zu erlernende und vielseitige Programmiersprache**. Sie wurde Anfang der 1990er-Jahre von Guido van Rossum entwickelt und erstmals 1991 veröffentlicht. Python zeichnet sich durch eine klare und gut lesbare Syntax aus, die es besonders für Einsteiger attraktiv macht.

Python gehört zu den sogenannten **höheren Programmiersprachen**, da es eine hohe Abstraktionsebene bietet und ermöglicht, Programme mit relativ wenig Code zu schreiben. Python verwendet eine klare und leicht verständliche Syntax, die sich an natürlicher Sprache orientiert. Es kann auf verschiedenen Betriebssystemen wie Windows, macOS und Linux ausgeführt werden. Python bietet eine umfangreiche Sammlung an Modulen und Bibliotheken für verschiedene Anwendungen: von Webentwicklung über Datenanalyse bis hin zu künstlicher Intelligenz.

Python wird in **vielen verschiedenen Bereichen** eingesetzt, darunter:

- Webentwicklung - leistungsfähige Webanwendungen
- Datenanalyse und wissenschaftliches Rechnen - Analyse großer Datenmengen
- Künstliche Intelligenz und maschinelles Lernen
- Automatisierung und Scripting - Python eignet sich hervorragend für kleine Skripte zur Automatisierung von Aufgaben
- Spieleentwicklung

Python ist nicht nur **besonders für Einsteiger geeignet**, sondern auch für professionelle Entwickler attraktiv, da es mächtige Werkzeuge für komplexe Anwendungen zur Verfügung stellt. Durch die große Community und die zahlreichen verfügbaren Ressourcen ist Python eine hervorragende Wahl für alle, die Programmieren lernen oder ihre Kenntnisse erweitern möchten.

Zusammenfassend lässt sich sagen, dass Python eine **vielseitige, benutzerfreundliche und leistungsfähige Programmiersprache** ist, die sich für zahlreiche Anwendungen eignet – von der einfachen Skripterstellung bis hin zur Entwicklung anspruchsvoller Softwareprojekte.

1.7. Geschichte von Python

Guido van Rossum schrieb 1996 über die Entstehung des Namens seiner Programmiersprache: "Vor über sechs Jahren, im Dezember 1989, suchte ich nach einem 'Hobby'-Programmier-Projekt, das mich über die Woche um Weihnachten beschäftigen konnte. Mein Büro ... war zwar geschlossen, aber ich hatte einen PC und sonst nichts vor. Ich beschloss mir einen Interpreter für die neue Skripting-Sprache zu schreiben, über die ich in der letzten Zeit nachgedacht hatte: ein Abkömmling von ABC, der UNIX/C-Hackern gefallen würde. Python hatte ich als Arbeitstitel für das Projekt gewählt, weil ich in einer leicht respektlosen Stimmung war (und ein großer Fan von Monty Python's Flying Circus)."

Guido van Rossum sagte in einem Interview: "Anfang der 80er Jahre habe ich an der CWI mit einem Team an der Sprache ABC gearbeitet. Ich hatte keine Ahnung wie ABC Python beeinflussen würde. Ich dachte an meine Erfahrungen und an den Frust mit ABC, und ich entschied mich, eine einfache Skriptsprache zu entwerfen. Sie sollte die Vorteile von ABC haben, nicht aber die Probleme/Nachteile. Es entstand eine einfache virtuelle Maschine, ein einfacher Parser, eine einfache Laufzeitumgebung und eine Syntax, die Einrückungen für die Gruppierung von Ausdrücken verwendet, und ein paar Datentypen: Dictionaries, Listen, Strings und Numbers/Integer."

Die folgenden **19** Leitsätze (PEP 20, auch: Zen of Python) beschreiben wichtige Grundideen der Programmiersprache Python:

ZEN of Python! (original)

Beautiful is better than ugly.

Explicit is better than implicit.

Simple is better than complex.

Complex is better than complicated.

Flat is better than nested.

Sparse is better than dense.

Readability counts.

Special cases aren't special enough to break the rules.

Although practicality beats purity.

Errors should never pass silently.

Unless explicitly silenced.

In the face of ambiguity, refuse the temptation to guess.

There should be one - and preferably only one - obvious way to do it.

Although that way may not be obvious at first unless you're Dutch.

Now is better than never.

Although never is often better than *right* now.

If the implementation is hard to explain, it's a bad idea.

If the implementation is easy to explain, it may be a good idea.

Namespaces are one honking great idea -- let's do more of those!

ZEN of Python! (übersetzt)

Schön ist besser als hässlich.

Explizit ist besser als implizit.

Einfach ist besser als komplex.

Komplex ist besser als kompliziert.

Flach ist besser als verschachtelt.

Sparsam ist besser als dicht.

Lesbarkeit zählt.

Sonderfälle sind nicht speziell genug, um die Regeln zu brechen.

Obwohl die Praktikabilität die Reinheit übertrifft.

Fehler sollten niemals schweigend vorübergehen. Es sei denn, sie werden ausdrücklich zum Schweigen gebracht.

Bei Mehrdeutigkeit: Widersteht der Versuchung zu raten.

Es sollte einen - und vorzugsweise nur einen - naheliegenden Weg geben, es zu tun.

Obwohl das anfangs nicht offensichtlich ist, es sei denn, man ist Holländer.

Jetzt ist besser als nie.

Obwohl nie oft besser ist als *gerade* jetzt.

Wenn die Umsetzung schwer zu erklären ist, ist es eine schlechte Idee.

Wenn die Umsetzung leicht zu erklären ist, kann es eine gute Idee sein.

Namespaces sind eine tolle Idee - lasst uns mehr davon machen!



